Three Separate Services — Clear Boundaries

Each service is independent: different protocol, different purpose, different client

SERVICE 1: Fleet Management (MQTT)

Fleet Management

Protocol: MQTT :1883

Server: Fleet Server (Docker)

Client: ESP32 devices

- **Heartbeat 30s** — uptime, heap, rssi
- **Auto-Register** — MAC, version, role
- **OTA Commands** — firmware push
- **Dead Detection** — 90s timeout → alert

Stored in: PostgreSQL + Grafana

ESP32 → MQTT → Fleet API → DB

SERVICE 2: Home Control (ESPHome API)

Home Assistant Control

Protocol: ESPHome Native API :6053

Server: HA Server (HA VM)

Client: ESP32 devices

- **Sensor Data** — temp, motion, door
- **Device Control** — lights, switches
- **Automations** — fall alert, emergency
- **OTA from HA** — ESPHome add-on

Real-time: <50ms, auto-discovery via mDNS

ESP32 → Native API → HA → Automation

SERVICE 3: Broadcast (REST Pub/Sub)

Broadcast Service

Protocol: REST API + FCM Pub/Sub

Server: Fleet Server (REST API)

Client: SmartHome iOS/Android App

- **POST /broadcast** — admin publishes msg
- **FCM Subscribe** — app subscribes to topic
- **Push Notification** — real-time to app
- **Delivery ACK** — app confirms receipt

Like Azure Event Hub — event-driven pub/sub

Admin → REST API → Push → Mobile App

Each Service — Different Protocol, Different Client, Different Purpose

Fleet (MQTT)

Protocol: MQTT :1883
Client: ESP32 devices
Purpose: Track, monitor, OTA update fleet

Home Control (API)

Protocol: ESPHome Native API :6053
Client: ESP32 devices
Purpose: Sensors, switches, automations

Broadcast (REST Pub/Sub)

Protocol: REST API + FCM push
Client: SmartHome iOS/Android app
Purpose: Community notifications to patients

IMPORTANT: These 3 services DO NOT overlap

Broadcast ≠ MQTT. ESP32 ≠ mobile app. HA ≠ fleet management. Each has its own protocol, client, and data.

Where Each Runs

Fleet Server
Fleet API + Mosquitto
Broadcast REST API

HA Server
Home Assistant VM
ESPHome add-on

ESP32 Device
→ MQTT to Fleet Server (MQTT)
→ API to HA Server (control)

Mobile App
→ REST API to Fleet Server → FCM push
(FCM topic subscriber)

Broadcast Service — REST Pub/Sub (Like Azure Event Hub)

Separate from MQTT and HA — Admin publishes via REST API, SmartHome app subscribes via WebSocket

Broadcast Architecture (Event-Driven Pub/Sub)

Uses Firebase Cloud Messaging (FCM) — single API for iOS + Android push

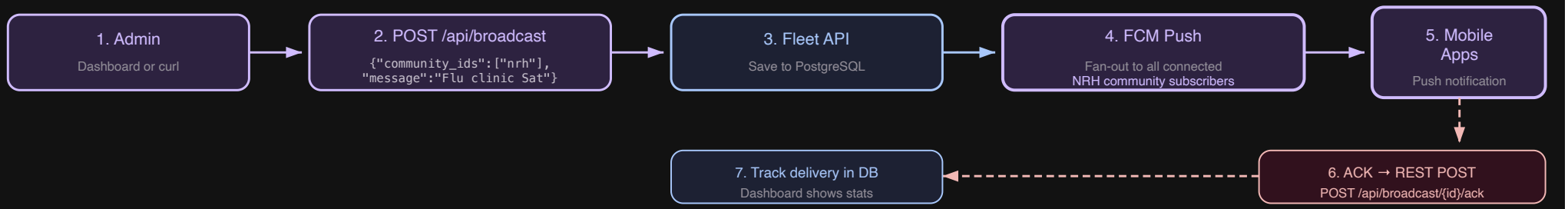
Publisher: Admin via REST API → FCM | Subscribers: iOS/Android apps via FCM topics

Protocol: REST API + FCM — NOT MQTT

REST API Endpoints

```
POST /api/broadcast/ → publish to community
GET /api/broadcast/ → list all broadcasts
GET /api/broadcast/{id} → detail + delivery stats
FCM topic subscribe (on app install)
```

Broadcast Message Flow — Step by Step



How SmartHome App Subscribes (Pub/Sub Pattern)

App subscribes to FCM topics on login. When admin sends broadcast, FCM pushes to all subscribed iOS + Android devices instantly.

If app is offline: FCM queues and delivers when device reconnects (built-in offline support).

Broadcast vs Fleet MQTT — Completely Separate

Fleet MQTT (Service 1)

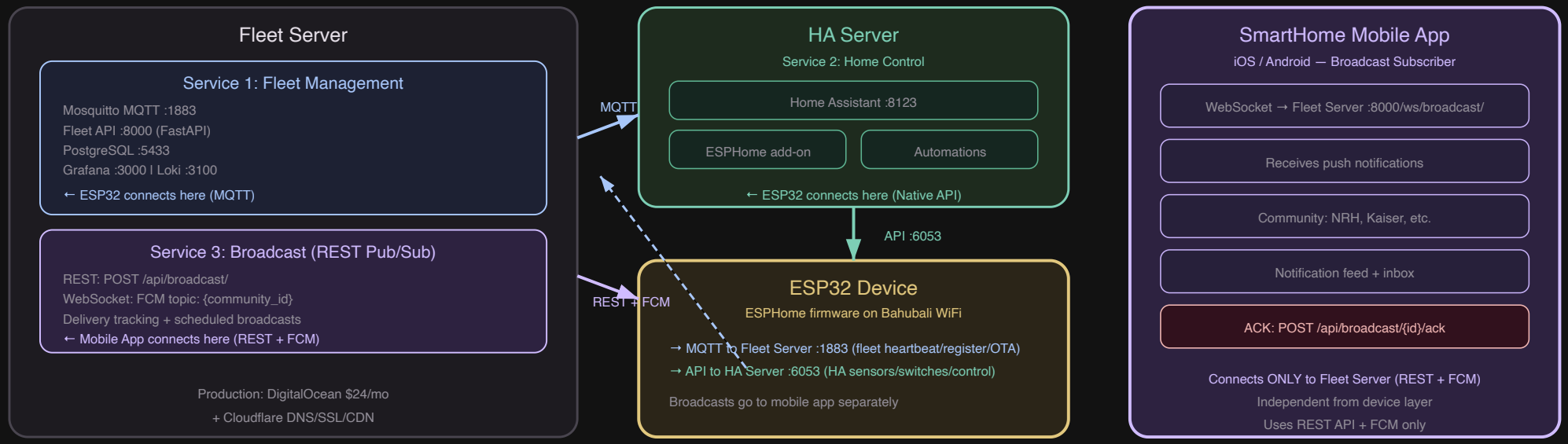
Protocol: MQTT :1883
Client: ESP32 hardware devices
Data: heartbeat, register, OTA commands
Direction: ESP32 → Server (mostly)
Frequency: every 30 seconds
Purpose: device health + firmware management

Broadcast REST (Service 3)

Protocol: REST API + FCM
Client: SmartHome iOS/Android app
Data: community notifications (text messages)
Direction: Server → App (push)
Frequency: on-demand (admin sends)
Purpose: patient communication by community group

Local Setup — Mac + HA Server + ESP32 + Mobile App

Three independent services, each with its own protocol and client



Protocol Separation — No Overlap

SERVICE	PROTOCOL	SERVER	CLIENT	DATA	PURPOSE
Fleet Mgmt	MQTT :1883	Fleet Server (Mosquitto)	ESP32 devices	Heartbeat, register, OTA	Track + update device fleet
Home Control	ESPHome API :6053	HA Server	ESP32 devices	Sensors, switches, automations	Local smart home control
Broadcast	REST API + FCM	Fleet Server	SmartHome mobile app	Community notifications	Patient communication